

AdaIN StyleKV: 基于自注意力KV融合与自适应归一化的扩散模型免训练风格迁移

姜俊吉 吕卓靓 杨沛文

电子科技大学计算机科学与工程（网安）学院

{junji.jiang, zhuoliang.lv, peiwen.yang}@uestc.edu.cn

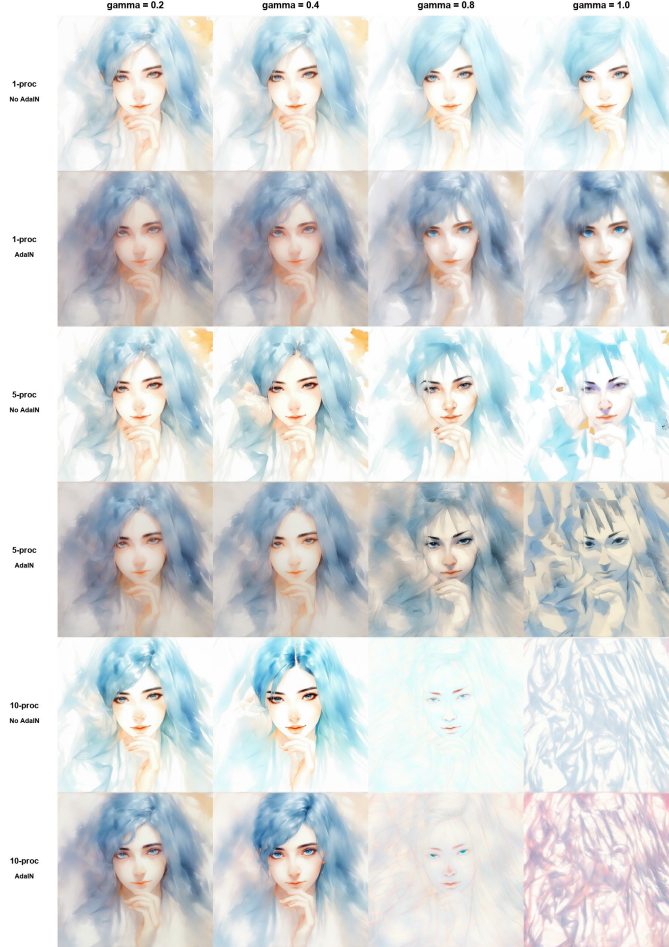


图 1. AdaIN StyleKV 风格迁移效果展示。横向为 γ 值从0.2到1.0逐步增大，纵向为1-proc、5-proc、10-proc分别搭配无AdaIN与有AdaIN的配置；其中5-proc + AdaIN为本文完整方法，在 $\gamma = 0.6$ 附近取得较优的风格强度与内容保真度平衡。

摘要

艺术风格迁移旨在将参考图像的视觉风格渲染到内容图像上，同时保留其语义结构。尽管扩散模型显著推动了该领域的发展，部分现有方法需要针对新的风格图像进行优化或反演，另一些免训练方法则仍面临结构保持、色彩一致性或强度控制不足的问题。为此，本文提出 *AdaIN StyleKV*，一个构建于预训练扩散模型之上的免训练风格迁移框架。该方法集成了以下关键组件：(1) *Canny ControlNet*，用于在生成过程中保持内容图像的边缘与轮廓结构；(2) 选择性自注意力KV融合机制，受 *Style Injection* 启发，在目标 *UNet block up_blocks.0.attentions.1* 内选择若干 *self-attention processor*，缓存风格图像的键值特征，并与内容图像的对应键值特征进行加权融合；(3) 跨模态 *AdaIN* 模块，用于在目标 *UNet block* 上融合已经嵌入的文本特征与图像特征，增强色彩风格迁移与氛围一致性。在 *MS COCO*（内容）与 *WikiArt*（风格）数据集上的组件消融、方法变体对比与参数敏感性分析表明，目标 *block up_blocks.0.attentions.1* 内 5 个 *self-attention processor* 的 *StyleKV* 配置在不启用 *AdaIN* 时取得最低 *Art-FID* (85.43)；相较于无 *AdaIN* 的 *StyleKV* 配置，完整方法在加入 *AdaIN* 后将 *LPIPS* 由 0.629 降至 0.599，在风格注入与感知变化之间形成更稳定的折中。此外，本文还实现了一个基于 *Gradio* 的交互式前端，支持用户实时调节风格强度、*ControlNet* 参数及 *AdaIN* 融合系数。

关键词：风格迁移；扩散模型；自注意力；自适应实例归一化；免训练；ControlNet

1. 引言

在计算机视觉与计算机图形学领域, 图像艺术风格迁移一直是一个引人入胜的研究方向。给定一幅内容图像和一幅风格图像, 风格迁移任务的目标是合成一幅输出图像, 该图像既能保留原始场景的语义结构与空间布局, 又能充分体现参考图像的视觉风格, 包括纹理、笔触和色彩调色板等要素。

风格迁移的发展经历了从非参数化纹理合成[1]、基于优化的神经网络方法[2]、前馈卷积网络[3,4]到生成对抗网络(GAN)[5,6,7]的范式演进。每一种范式都带来了生成速度与迁移质量的显著提升, 但同时也存在固有局限: GAN类方法通常局限于训练阶段所见过的风格域, 难以泛化至任意风格; 而基于优化的方法计算开销高昂, 难以满足实时应用需求。

近年来, 扩散模型(Diffusion Models)[8,9,10]的兴起极大地推动了生成建模领域的进展。Stable Diffusion[11]及其后续版本凭借潜在扩散机制与交叉注意力模块, 能够从文本提示生成高分辨率、语义丰富的图像。受此启发, 研究者将扩散模型进一步拓展至风格迁移任务[12,13,14,15,16,17]。然而, 现有基于扩散的方法仍面临若干显著挑战。首先, InST[12]和StyleDiffusion[13]等方法需要对每种新的风格图像进行基于梯度的微调或文本反演, 单幅图像的风格适应往往耗时较长, 限制了其在大规模或实时场景中的应用。其次, 部分方法在应用于潜在扩散模型时难以可靠地保持原始图像的空间布局与语义结构, 可能在迁移纹理的同时改变内容的几何形态。再者, 自注意力操作虽然能够有效迁移局部纹理模式, 但往往忽略全局颜色统计信息, 导致输出图像在色调上缺乏整体协调性。此外, 大多数现有方法缺少在风格强度与内容保真度之间进行灵活调节的机制, 限制了用户的创作自由度。

针对上述挑战, 本文提出AdaIN StyleKV——一个构建于预训练扩散模型之上的免训练风格迁移框架, 无需额外微调或文本反演, 并在本文实验设置下实现较好的风格化效果。本文研究最初从一个较直接的naive AdaIN原型出发: 在SDXL和Canny ControlNet结构约束下, 结合InstantStyle的有效块选择经验, 将AdaIN用于目标UNet block中的文本与图像条件特征调制, 目标block数量可配置为1个或2个。该原型能够改善色彩协调性, 但对局部纹理和笔触模式的迁移仍不充分。基于这一观察, 我们进一步引入受Style Injection[16]启发的选择性KV融合机制, 形成最终的AdaIN StyleKV框架。在结构保持方面, 该框架引入Canny边缘检测引导的ControlNet, 通过在生成过程中保留内容图像的边缘与轮廓结构, 有效增强语义内容的稳定性。在纹理迁移方面, 本文所有proc开头的StyleKV实验均固定在目标block `up_blocks.0.attentions.1` 内进行KV融合, 并仅改变该block内参与融合的self-attention processor数量; 我们在这些processor中缓存风格图像的键和值特征, 并与内容图像的对应特征进行加权融合, 从而实现局部纹理模式的有效迁移。在色彩一致性方面, 我们将AdaIN作为方法起点, 并扩展为文本与图像条件之间的跨模态特征融合模块, 对已经嵌入的文本特征和图像特征进行自适应归一化融合, 增强色彩风格迁移与整体氛围的和谐统一。此外, 该框架还通过引入融合系数 γ , 为用户提供了直观且连续的风格强度控制能力, 使其能够在风格化程度与内容保留之间进行灵活取舍。

我们在MS COCO[18]内容数据集和WikiArt[19]风格数据集上对本文方法进行了组件消融、方法变体

对比与参数敏感性分析。定量评估涵盖Art-FID[20]、FID[21]和LPIPS[22]等多个维度。实验结果表明, AdaIN StyleKV在风格保真度与内容保留之间取得了较优平衡; 其中, `up_blocks.0.attentions.1` 内5个self-attention processor的StyleKV配置在不启用AdaIN时取得最低Art-FID(85.43), 完整方法则在加入AdaIN后将LPIPS降至0.599, 说明其感知变化更温和。此外, 我们还通过多组对比实验, 系统揭示了不同配置(如目标block数量、`up_blocks.0.attentions.1` 内KV融合processor数量、融合系数 γ 、AdaIN模块的有无)对风格迁移效果的影响规律。为提升方法的实用性与可交互性, 本文还实现了一个基于Gradio的交互式前端演示系统, 支持用户实时调整风格强度、ControlNet参数和AdaIN融合系数。

2. 相关工作

2.1. 神经风格迁移

神经风格迁移由Gatys等人[2]开创, 他们使用VGG[23]特征的Gram矩阵匹配通过迭代优化实现风格迁移。后续工作主要聚焦于提升速度与泛化能力: Johnson等人[24]引入了基于感知损失的前馈图像变换网络, 实现了实时风格迁移; Huang和Belongie[4]提出了自适应实例归一化(AdaIN), 实现了前馈式的任意风格迁移。其他值得关注的方法包括基于注意力的方法[25]、基于Transformer的方法[26]、基于可逆流的方法[27]以及双重风格学习框架[28]。尽管这些方法在速度上相比Gatys等人的优化式方法有显著提升, 但在处理高度抽象或非写实风格时仍存在局限。

2.2. 基于GAN的风格迁移

生成对抗网络在不成对图像翻译任务上取得了显著进展。CycleGAN[5]引入了循环一致性损失, 使得无需配对数据即可实现跨域图像转换; StarGAN[6]进一步将其扩展为多领域转换框架, 能够以单一模型处理多个目标域之间的翻译任务。此外, 以StyleGAN[7]及其变体[29,30]为代表的高质量生成模型, 提供了精细且可解释的潜在空间控制能力, 被广泛用于领域内的图像编辑与属性操作。但需要指出的是, StyleGAN本质上是一个无条件生成模型, 其设计目标并非跨域翻译; 在应用于领域外风格迁移时, 往往需要额外的微调或特征解耦策略, 难以直接实现与CycleGAN等翻译方法等价的任意风格泛化。

2.3. 基于扩散模型的风格迁移

扩散模型已成为高质量图像生成的主导范式。其中, 潜在扩散模型(LDM)[11]通过将图像压缩至低维潜在空间, 实现了高效的训练与推理。针对风格迁移任务, 研究者基于扩散模型提出了多种方法:

InST[12]采用文本反演技术将风格图像嵌入至文本向量空间, 能够较好地保留风格特征, 但需要对每种风格单独进行优化。StyleDiffusion[13]引入基于CLIP的损失函数与微调策略, 以实现内容与风格的解耦。DiffStyle[14]是一种运行于扩散模型H空间的免训练方法, 然而将其应用于Stable Diffusion时可能会改变图像的语义布局。Style Injection[16]通过将自注意力层中的键值特征替换为风格图像的特征, 实现了免训练的风格迁移, 并在一定程度上保留内容结构, 但缺少类似ControlNet的显式结构约束, 且颜色迁移能力有限。InstantStyle[17]发现仅有特定的UNet块对风格迁移起关键作用, 并通过对这些块进行

特征嵌入实现了高效的风格注入。StyleStudio[31]进一步探索了文本驱动风格迁移中对风格元素的选择性控制。

方法	免训练	结构保持	色彩迁移	强度可控
InST[12]	×	×	✓	×
StyleDiffusion[13]	×	×	✓	✓
DiffStyle[14]	✓	×	×	×
Style Injection[16]	✓	×	×	×
InstantStyle[17]	✓	×	×	×
AdaIN StyleKV (本文)	✓	✓	✓	✓

表 1. 本文方法与代表性风格迁移方法的对比。免训练：无需梯度优化；结构保持：是否显式引入边缘、深度等空间条件约束；色彩迁移：是否专门处理全局色彩信息；强度可控：是否提供显式连续控制参数。

表 1总结了本文方法与代表性方法的异同。本文方法以AdaIN特征调制为起点，并吸收Style Injection的KV替换思想和InstantStyle的有效block选择经验，同时结合ControlNet的结构控制能力。在此基础上，我们将直接替换改为在目标block `up_blocks.0.attentions.1` 内的self-attention processor中进行可控的KV加权融合，从而在纹理迁移与色彩和谐之间取得了更好的平衡。

2.4. 扩散模型中的条件控制

ControlNet[15]提出了一种神经架构，使用零卷积层向扩散模型添加空间条件控制（边缘、深度、姿态等）。它将原始模型锁定并创建一个可训练的副本，确保训练不会破坏预训练特征。本文集成ControlNet的Canny边缘版本，以在执行风格迁移的同时保持内容图像的结构轮廓。需要注意的是，所使用的ControlNet须与基础模型版本匹配。本文全部实验均基于Stable Diffusion XL (SDXL) 架构，采用其对应的Canny ControlNet权重 `diffusers/controlnet-canny-sdxl-1.0`；SD1.5架构则需改用 `lillyasviel/control_v11p_sd15_canny` 等匹配权重。选择SDXL作为基础模型，是为了利用其更高的生成分辨率与更强的语义表达能力。

3. 方法

本章详细阐述我们提出的AdaIN StyleKV方法。我们首先介绍整体框架，随后依次介绍选择性KV融合、跨模态AdaIN与ControlNet集成等核心组件。

3.1. 整体框架

我们的方法基于Stable Diffusion + ControlNet (Canny)，并在此基础上结合跨模态AdaIN与选择性KV融合机制。整体框架分为以下五个阶段：

阶段一：图像预处理与编码。给定内容图像和风格图像，我们首先提取内容图像的Canny边缘图，作为ControlNet的空间条件输入。然后，使用VAE编码器将内容图像和风格图像编码到潜空间。

阶段二：DDIM反演与潜变量轨迹构建。我们使用DDIM采样器对内容图像和风格图像分别进行反演，得到它们在每个时间步的潜变量轨迹，为后续跨模态AdaIN融合和逐步KV特征缓存提供输入。

阶段三：文本与风格条件的跨模态AdaIN。在DDIM反演的每个时间步进行条件注入时，我们不采用基于适配器 (Adapter-Based) 的方式，而是在InstantStyle指出的

目标UNet block上，使用自适应实例归一化 (AdaIN) 对已嵌入的文本特征与图像特征进行跨模态融合。

阶段四：风格特征缓存与内容图像的KV融合。我们先确定目标UNet block `up_blocks.0.attentions.1`，再缓存该block内选定self-attention processor中的风格图像K、V特征，并在内容图像的反向扩散生成过程中于对应processor内将其与内容特征融合。在每个时间步：(1) 在风格图像对应潜变量的U-Net前向过程中，记录`up_blocks.0.attentions.1`内选定self-attention processor产生的K/V特征；(2) 在内容潜变量去噪过程中，对于该block内选定的self-attention processor，将当前计算得到的内容K和V与缓存的风格K和V进行加权融合；(3) 使用融合后的K/V进行自注意力计算；(4) 对于非目标processor，保持原始的K/V不变。

阶段五：解码与输出。反向扩散完成后，得到生成图像的潜变量。将其输入VAE解码器，得到最终的风格化图像。

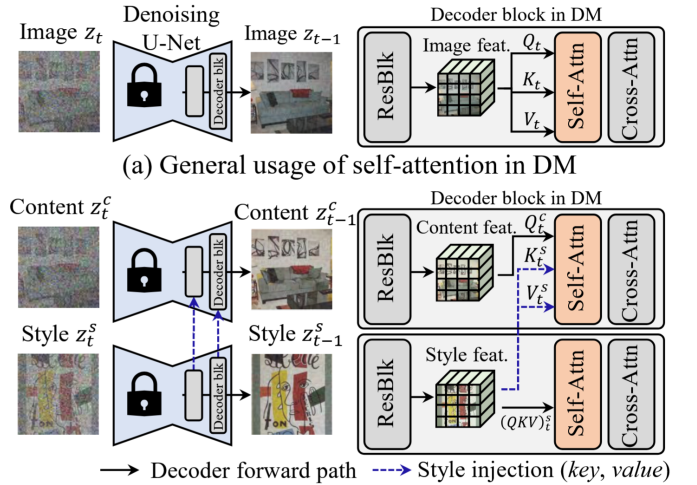


图 2. AdaIN StyleKV整体方法流程框架图。给定内容图像与风格图像，首先使用VAE编码器将二者编码到潜空间，同时将内容图像的Canny边缘图作为ControlNet空间条件；在DDIM反演阶段获得内容图像与风格图像的潜变量轨迹；文本条件与图像条件经跨模态AdaIN进行特征融合；反向扩散过程中缓存风格图像在目标block `up_blocks.0.attentions.1` 内选定self-attention processor中的K/V特征，并与内容图像对应processor的K/V进行加权融合，最终经VAE解码器生成风格化图像。

图 2以可视化方式总结了上述五阶段流程，清晰展示了内容图像与风格图像在潜空间中的交互过程。

3.2. 选择性KV融合

3.2.1 自注意力机制回顾

在U-Net中，自注意力层的计算方式如下。给定残差块输出的特征 X ，首先通过三个投影矩阵得到查询 Q 、键 K 和值 V ：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (1)$$

然后，通过缩放点积注意力计算输出：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2)$$

其中 d_k 是查询向量的维度。在标准的生成过程中， Q 、 K 、 V 都来自同一张图像的特征。但在风格迁移任务中，我

们通过加权融合的方式将内容图像的 K 、 V 与风格图像的 K 、 V 相结合。通常可近似理解为, 内容侧 Q 有助于保持当前潜变量的空间组织, 而融合后的 K/V 会影响注意力检索到的纹理与外观模式。

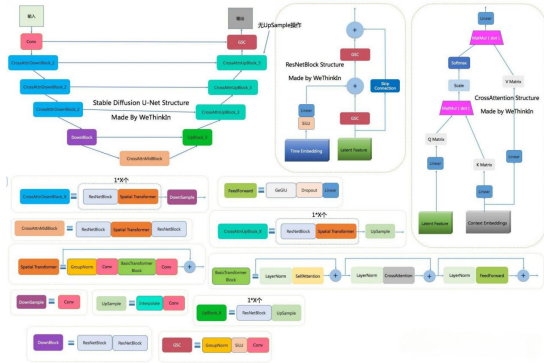


图 3. Stable Diffusion 中 U-Net 架构示意图。U-Net 由下采样编码器、中间块和上采样解码器三部分组成, 每个部分包含多个 ResNet 残差块和自注意力层。本文方法先依据 InstantStyle 选择特定解码器目标 block `up_blocks.0.attentions.1`, 再仅在该 block 内的目标 self-attention processor 中进行 KV 融合 (红色标记), 其余注意力处理器保持原有计算不变。

图 3 展示了 U-Net 的整体架构及各层中自注意力模块的分布, 为后续 KV 融合的目标注意力处理器选择提供了结构参照。

3.2.2 KV 加权融合

受到 Style Injection[16] 的启发, 我们对目标 block `up_blocks.0.attentions.1` 内选定 self-attention processor 中的 K/V 进行操作。不同于直接用风格图像特征替换内容图像特征, 本文采用加权融合形式:

$$K_{\text{fused}} = (1 - \gamma)K_{\text{content}} + \gamma K_{\text{style}} \quad (3)$$

$$V_{\text{fused}} = (1 - \gamma)V_{\text{content}} + \gamma V_{\text{style}} \quad (4)$$

其中 $\gamma \in [0, 1]$ 是风格融合强度系数。这种设计具有以下优势: (1) 可调节: $\gamma = 0$ 完全保留内容 K/V , $\gamma = 1$ 完全使用风格 K/V , 中间值平滑过渡; (2) 免训练: 仅通过线性插值实现风格注入, 无需梯度优化; (3) 稳定性: 加权融合比直接替换更平滑, 减少生成突变。

3.2.3 目标 block 与注意力处理器的选择

遵循 InstantStyle[17] 关于有效区域选择的发现, 我们首先确定对风格迁移更敏感的目标 UNet block。本文所有 StyleKV 实验均固定在 `up_blocks.0.attentions.1` 这一目标 block 内进行, 而不是跨多个 block 替换 K/V 。在该 block 内部, 再选择少数 self-attention processor 参与 StyleKV 融合。实验测试了三种 processor 数量配置:

- 1 个处理器: 在 `up_blocks.0.attentions.1` 内仅选择 1 个 `attn1.processor` 进行 KV 融合
- 5 个处理器 (默认): 在 `up_blocks.0.attentions.1` 内选择 5 个 `attn1.processor` 进行 KV 融合
- 10 个处理器: 在 `up_blocks.0.attentions.1` 内选择 10 个 self-attention processor 进行 KV 融合 (覆盖该 block 内更多目标位置)

实验表明, 在 `up_blocks.0.attentions.1` 内选择 5 个 self-attention processor 的配置在风格迁移效果和内容保留之间取得了较优平衡。

表 2. 早期 naive AdaIN 原型中的目标 block 配置。1blk/2blk 表示 AdaIN 作用的 UNet 目标 block 数量; 与此不同, StyleKV 实验固定在 `up_blocks.0.attentions.1` 内改变 self-attention processor 数量。

配置	AdaIN 作用的目标 UNet block	对应 AdaIN 变体
1blk	<code>up_blocks.0.attentions.1</code>	V1: adain_1blk
2blk	<code>up_blocks.0.attentions.1</code> ; <code>down_blocks.2.attentions.1</code>	V2: adain_2blk

3.3. 文本与图像条件的跨模态 AdaIN

在艺术风格迁移中, 色调与整体氛围占据风格信息的重要部分。然而, 我们观察到仅通过 KV 融合进行风格注入往往难以准确捕捉给定风格的全局色彩统计——纹理和局部图案虽然成功迁移, 但文本条件与图像条件之间仍可能存在语义和色彩分布的不一致。

不同于需要训练新适配器的方法, 本文不额外训练新的适配器参数, 而是在 DDIM 反演和条件注入过程中, 在目标 UNet block 上对已经嵌入的文本特征与图像特征使用 AdaIN 进行跨模态融合。本文中的风格图像嵌入由预训练图像编码器提取, 实验实现中使用 IP-Adapter 配置中的 image encoder; AdaIN 以该图像嵌入的通道统计量调制文本条件特征, 使文本语义引导与参考风格图像的视觉统计保持一致。

给定文本嵌入特征 f_{text} 和风格图像嵌入特征 f_{style} , 跨模态 AdaIN 定义为:

$$f_{\text{adain}} = \sigma(f_{\text{style}}) \left(\frac{f_{\text{text}} - \mu(f_{\text{text}})}{\sigma(f_{\text{text}})} \right) + \mu(f_{\text{style}}) \quad (5)$$

其中 $\mu(\cdot)$ 和 $\sigma(\cdot)$ 分别表示通道维度的均值和标准差。代码实现中进一步引入超参数 α 和 β , 其中 α 控制原文本特征与 AdaIN 特征的线性混合, β 用于缩放 AdaIN 分支中的平移项:

$$f_{\text{adain}}^{(\beta)} = \sigma(f_{\text{style}}) \left(\frac{f_{\text{text}} - \mu(f_{\text{text}})}{\sigma(f_{\text{text}})} \right) + \beta \mu(f_{\text{style}}) \quad (6)$$

$$f_{\text{out}} = (1 - \alpha)f_{\text{text}} + \alpha f_{\text{adain}}^{(\beta)} \quad (7)$$

本文实验默认 $\alpha = 1.0$ 、 $\beta = 1.0$, 因此退化为式 5 所示的标准 AdaIN 形式。该操作保留文本嵌入中的语义方向, 同时将其通道统计量对齐到风格图像嵌入, 从而在不依赖额外适配器训练的情况下增强文本条件与视觉风格条件的一致性。

3.4. ControlNet 集成

为保持内容图像的空间结构, 我们集成了 Canny ControlNet。ControlNet 接收 Canny 边缘图作为条件输入, 其输出被加到 U-Net 的跳跃连接和中间块上。其设计优势包括: (1) 零卷积层保证训练起始稳定性; (2) 在 ControlNet 原始训练范式中锁定原模型参数, 仅训练 ControlNet 分支; (3) 即插即用, 无需修改原模型架构。本文直接使用与基础模型版本匹配的预训练 ControlNet 权重, 无需进一步训练。

3.5. 推理阶段的条件生成

本文不重新训练扩散模型，而是在预训练Stable Diffusion的推理过程中注入Canny结构条件、跨模态AdaIN条件和选择性KV融合。给定时间步 t 、文本提示 c 、Canny条件 c_{canny} 以及当前潜变量 z_t ，预训练噪声预测网络可表示为：

$$\hat{\epsilon} = \epsilon_{\theta}(z_t, t, c, c_{\text{canny}}) \quad (8)$$

其中 ϵ_{θ} 的参数保持冻结。AdaIN StyleKV仅在目标UNet block的条件特征，以及`up_blocks.0.attentions.1`内选定self-attention processor的K/V特征上进行推理时调制，随后按照上述五阶段流程生成风格化图像。

4. 实验与结果

4.1. 实验设置

4.1.1 数据集

内容数据集：使用MS COCO 2017数据集[18]中的5000张内容图像，每张图像配有对应的文本提示词，用于辅助扩散模型理解画面内容。

风格数据集：使用WikiArt数据集[19]中的5000张风格图像，涵盖印象派、立体派、抽象派等不同艺术风格。

图像配对：5000张内容图像与5000张风格图像按顺序一一对应配对，共5000对。

数据规模：共计生成5000张风格化图像用于定量评估。

4.1.2 评估指标

需要说明的是，本文三项指标均以“越低越好”的方向使用，但各自衡量的对象不同：Art-FID对齐风格域，FID对齐内容域，LPIPS度量逐对感知改变。由于风格迁移本质上是内容一风格的权衡，三者通常无法同时最优，需联合解读。

Art-FID：专门针对艺术图像生成的综合指标[20]。在本文实现中，Art-FID用于衡量生成图像集合与目标风格图像集合之间的分布距离，数值越低表示生成结果与风格域越接近。这是本文的主要评估指标。

FID (Fréchet Inception Distance) [21]：衡量生成图像与内容图像在InceptionV3特征空间上的分布距离。在本任务中，较低的FID仅间接反映生成结果与内容域的整体分布接近程度，不能单独作为结构保持指标。

LPIPS (Learned Perceptual Image Patch Similarity) [22]：基于预训练深度网络特征差异，逐对计算风格化图像与内容图像的感知相似度。在相同任务设置下，

较低LPIPS通常表示相对内容图的感知变化较小。由于风格化必然引入感知变化，本文将LPIPS与FID、Art-FID联合解读，而不将其单独作为风格迁移质量指标。

4.1.3 硬件配置与超参数

实验在AutoDL平台租用NVIDIA V100-32GB GPU进行。硬件与超参数配置分别如表 3和表 4所示。

表 3. 硬件配置与环境。

配置项	参数
GPU	NVIDIA V100 32GB × 1
加速库	x-formers
采样步数	50步 (DDIM)
分辨率	512×512 (中心裁剪)
生成总数	5000张

表 4. 超参数设置。 $\gamma = 0.6$ 为实验确定的较优权衡点，详见第4.4节敏感性分析；KV融合默认在`up_blocks.0.attentions.1`内选择5个self-attention processor；AdaIN融合强度默认1.0。

参数	默认值	说明
γ	0.6	风格融合强度系数
<code>up0.attn1</code> 内proc数	5	KV融合processor数量
<code>adain_alpha</code>	1.0	AdaIN融合强度
<code>adain_beta</code>	1.0	AdaIN融合偏置
CFG scale	7.5	无分类器引导强度
DDIM步数	50	反向扩散采样步数

4.2. 方法变体设计

为系统评估各组件的贡献，我们设计了7组方法变体（表 5）。V1-V5用于评估早期naive AdaIN原型及ControlNet的影响，其中1blk/2blk表示AdaIN作用的目标UNet block数量，其具体block如表 2所示；V6-V7进一步在`up_blocks.0.attentions.1`内引入5个self-attention processor的StyleKV融合，用于比较有无AdaIN时的风格注入与感知变化差异。需要强调的是，所有proc开头的变体均只在该block内改变processor数量。

需要说明的是，本文仅将单一因素发生变化的对比称为组件消融实验：V6与V7用于分析AdaIN模块的影响，V1与V5用于分析ControlNet结构约束的影响。V1/V2与V3/V4主要反映早期naive AdaIN原型中目标block数量变化带来的配置差异； γ 取值变化属于参数敏感性分析；1-proc、5-proc与10-proc结果用于定性观察同一目标block

Quantitative Comparison Across Evaluation Metrics

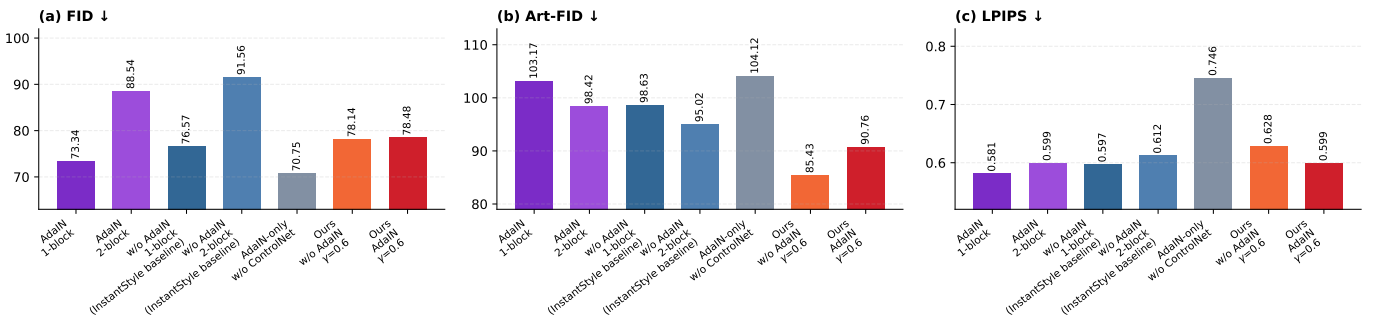


图 6. 7组方法变体在FID、Art-FID和LPIPS三项指标上的定量对比，三项指标均为越低越好。FID反映生成结果与内容域的分布距离，Art-FID反映与目标艺术风格域的接近程度，LPIPS反映感知层面的图像变化幅度。V6 (Ours w/o AdaIN) 取得最低Art-FID，风格注入更强；相较于V6，完整方法V7 (Ours AdaIN, $\gamma = 0.6$) 降低LPIPS，感知变化更温和。

内StyleKV目标processor数量的影响。因此，本文实验整体由组件消融、方法变体对比、参数敏感性分析和配置定性分析共同组成，而非将所有结果都归为严格消融。

表 5. 7组方法变体设计。

编号	变体名称	ControlNet	作用范围	AdaIN	γ	KV融合
V1	adain_1blk	✓	1 block	✓	—	×
V2	adain_2blk	✓	2 blocks	✓	—	×
V3	noadain_1blk	✓	1 block	×	—	×
V4	noadain_2blk	✓	2 blocks	×	—	×
V5	adain_no_ctrl	×	1 block	✓	—	×
V6	proc05_no_adain	✓	5 proc	×	0.6	✓
V7	proc05_adain	✓	5 proc	✓	0.6	✓

图 4和图 5给出了不同内容-风格配对下的可视化对比，用于直观观察各变体在风格注入强度、色彩协调性与内容保持上的差异。



图 4. “大叔”内容图上的不同方法输出横向对比。图中上方标签标明对应方法配置，用于直观比较各配置对风格注入强度与内容保持效果的影响。



图 5. 两组不同内容-风格配对下的方法变体对比。

4.3. 定量结果

表 6展示了7组变体在5000张生成图像上的定量评估结果。

表 6. 7组变体在5000张生成图像上的定量评估结果。FID衡量与内容域的分布距离（越低表示整体特征分布越接近原始内容）；Art-FID衡量与风格域的分布距离（越低表示风格相似度越高）；LPIPS衡量感知改变程度（风格化必然引入感知变化，适中的LPIPS值表示在内容可辨识前提下的有效风格迁移）。粗体为完整方法（V7）的结果。

变体名称	FID↓	Art-FID↓	LPIPS
V1 (adain_1blk)	73.34	103.17	0.581
V2 (adain_2blk)	88.54	98.42	0.599
V3 (no_adain_1blk)	76.57	98.63	0.597
V4 (no_adain_2blk)	91.56	95.02	0.612
V5 (adain_no_ctrl)	70.75	104.12	0.746
V6 (proc05_no_adain)	78.14	85.43	0.629
V7 (proc05_adain)	78.48	90.76	0.599

分析：V6 (proc05_no_adain) 取得最低 Art-FID (85.43)，说明在 `up_blocks.0.attentions.1` 内 5 个 self-attention processor 的 StyleKV 配置下，不使用 AdaIN 时风格纹理注入更强；但其 LPIPS 升至 0.629，表明感知变化也更明显。完整方法 V7 (proc05_adain) 加入 AdaIN 后，Art-FID 为 90.76，风格注入略弱，但 LPIPS 降至 0.599；结合定性结果，说明 AdaIN 有助于降低感知变化，并可能改善整体色彩协调性。早期 naive AdaIN 基线 (V1、V2) 能够改善色彩协调性，但 Art-FID 仍高于 `up_blocks.0.attentions.1` 内 5 个 processor 的 StyleKV

配置，说明仅依赖 AdaIN 条件调制不足以充分迁移局部纹理与笔触风格。V5 (无 ControlNet) 的 FID 最低 (70.75) 但 LPIPS 最高 (0.746)，说明虽然生成图像整体特征接近内容域，但局部内容结构丢失严重——这与 ControlNet 去除后缺乏显式结构约束的预期一致。V2 相对 V1、V4 相对 V3 改变的是 naive AdaIN 的目标 block 数量，而非 KV 融合 processor 数量；结果显示更多目标 block 会增强条件调制影响，但也可能带来更明显的内容结构改变。

为更直观地比较各方法在不同评价维度上的相对表现，我们将 FID、Art-FID 和 LPIPS 三项指标分别转换为排名 (1 为最优，7 为最差)，结果如图 7 所示。可以看到，不同变体在三个指标上呈现明显的取舍关系：AdaIN-only w/o ControlNet 在 FID 上排名最高，但在 Art-FID 和 LPIPS 上排名较低，说明缺少结构约束会带来更明显的感知变化；Ours w/o AdaIN 在 Art-FID 上排名最高，但 LPIPS 排名较低；完整方法 Ours AdaIN $\gamma = 0.6$ 在 LPIPS 上优于无 AdaIN 版本，体现了 AdaIN 对感知变化和色彩协调性的稳定作用。

Ablation Rank Heatmap (lower rank is better)

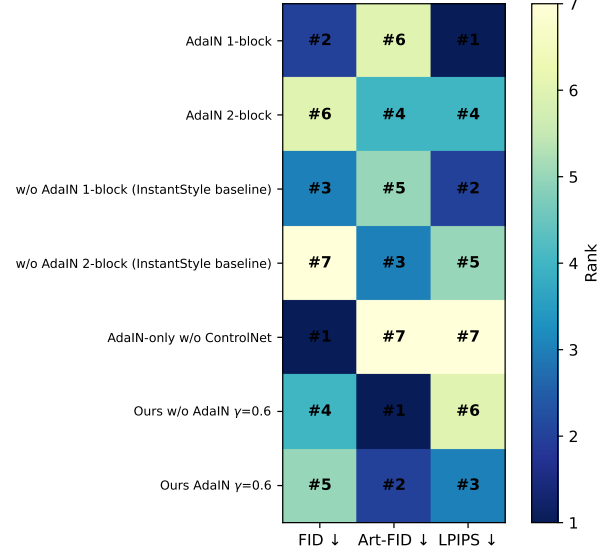


图 7. 7组方法变体在FID、Art-FID和LPIPS三项指标上的排名热力图，排名越低表示表现越好。

图 4和图 8展示了各变体的可视化对比结果。

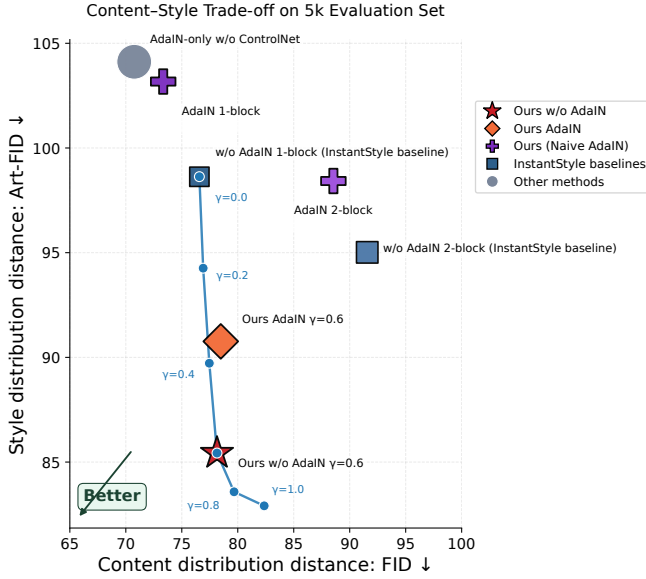


图 8. 各方法变体在FID与Art-FID上的内容-风格权衡对比。

4.4. Gamma强度系数敏感性分析

为分析风格融合强度系数 γ 对生成结果的影响，本文在固定`up_blocks.0.attentions.1`内5个self-attention processor、AdaIN与ControlNet配置的条件下，仅改变 γ 的取值，并测试 $\gamma \in \{0.0, 0.2, 0.4, 0.6, 0.8, 1.0\}$ 时的生成效果。实验结果如表 7所示。

表 7. γ 强度系数敏感性分析结果(`up_blocks.0.attentions.1`内5个self-attention processor配置, 固定AdaIN + ControlNet)。 $\gamma = 0$ 时主要保留内容特征，风格注入较弱； $\gamma = 1$ 时风格化程度最高，但内容改变也更明显。加粗表示本文默认配置，而非单项指标最优。

γ	FID↓	Art-FID↓	LPIPS
0.0	76.57	98.63	0.597
0.2	76.91	94.26	0.606
0.4	77.46	89.72	0.617
0.6	78.48	90.76	0.599
0.8	79.67	83.58	0.655
1.0	82.36	82.91	0.697

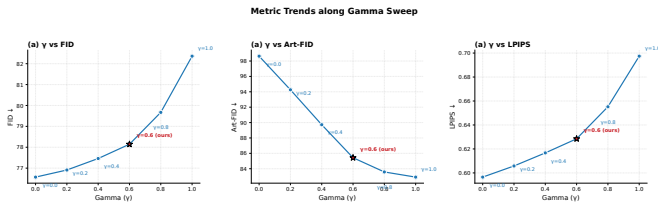


图 9. γ 取值对FID、Art-FID和LPIPS的影响。随着 γ 增大，风格保真度整体提升但内容变化也增加， $\gamma = 0.6$ 为本文采用的默认折中点。

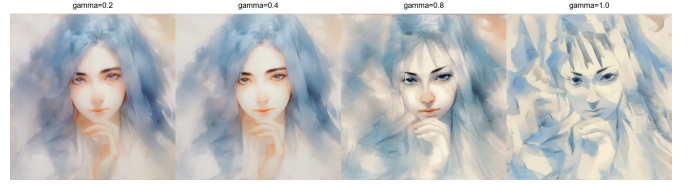


图 10. 在采用AdaIN的`up_blocks.0.attentions.1`内5个self-attention processor配置（本文方法）下不同 γ 值的风格迁移效果。从左至右 $\gamma = 0.2, 0.4, 0.8, 1.0$ ，可清晰观察风格纹理随 γ 增大而逐步增强的渐变过程。

结论：随着 γ 逐渐增大，风格纹理和色彩表现整体增强，生成结果与目标艺术风格分布更加接近；与此同时，内容保真度和细粒度结构也更容易受到影响。具体而言，较小 γ 更偏向内容保持，较大 γ 更偏向风格注入。结合定量指标与图 9和图 10中的视觉结果，本文选择 $\gamma = 0.6$ 作为更偏向内容稳定性的默认折中点。

4.5. StyleKV目标processor数量定性分析

表 2已经说明，1blk/2blk属于早期naive AdaIN原型中的目标block配置；而proc开头的实验关注的是StyleKV内部的目标block配置。两者不是同一消融维度，因此本节不再将它们作为同一组定量消融进行比较。为观察StyleKV内部processor数量对风格注入强度和保持的影响，本文固定目标block为`up_blocks.0.attentions.1`，并仅改变该block内参与K/V融合的self-attention processor数量，进行定性可视化分析。

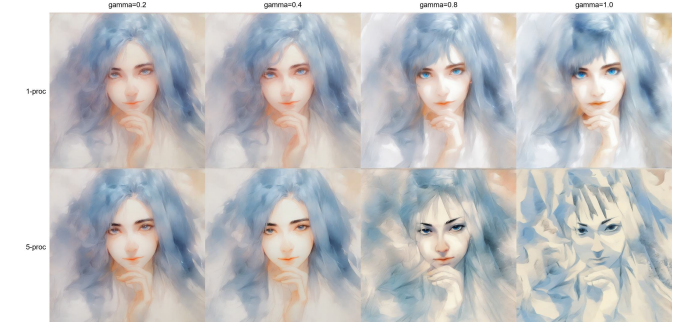


图 11. 在采用AdaIN的`up_blocks.0.attentions.1`内1个与5个self-attention processor配置在不同 γ 值下的风格迁移效果对比。上行为1-proc StyleKV配置，下行为5-proc StyleKV配置（本文方法）。更多目标processor带来更强的风格注入（下行风格纹理更明显），在展示样例中仍能保持较好的内容结构。

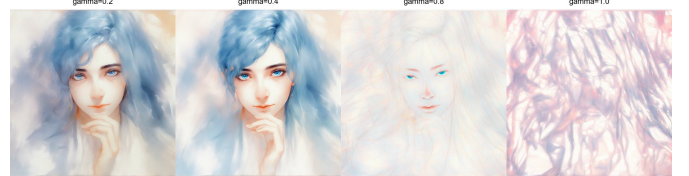


图 12. 在采用AdaIN的`up_blocks.0.attentions.1`内10个self-attention processor配置下不同 γ 值的风格迁移效果。与同一block内5-proc配置相比，10-proc在相同 γ 下风格注入更强，但内容细节的保持略逊于5-proc配置。

观察：图 11表明，在同一目标block内，5-proc配置相比1-proc配置能够带来更明显的纹理迁移，尤其在较高 γ 值下风格特征更充分。图 12进一步显示，10-proc配置会继续增强风格化程度，但在高 γ 下更容易削弱面部细节和局部结构。综合视觉效果与计算开销，本文将`up_blocks.0.attentions.1`内5个processor作为默认

StyleKV配置。

4.6. 定性结果分析

4.6.1 不同block/processor配置与AdaIN的效果对比

表 8对比了四种naive AdaIN基础配置的生成结果。图 13进一步展示了AdaIN模块在1个目标block配置下对色彩协调性的影响。

表 8. 不同目标block数量与AdaIN配置的定性对比。使用AdaIN的配置色彩协调性更优、风格覆盖更均匀；不使用AdaIN的配置风格迁移更强但内容结构退化更明显。

变体	block数	观察结果
adain_1blk	1	风格注入集中, 纹理迁移明显
adain_2blk	2	风格覆盖均匀, 色彩更协调
no_adain_1blk	1	风格迁移较强, 内容保留略逊
no_adain_2blk	2	风格注入增强, 结构更模糊



图 13. AdaIN模块对风格迁移效果的影响（1个目标block配置）。上行为使用AdaIN的生成结果，下行为不使用AdaIN的结果。AdaIN模块显著提升了色彩协调性与整体氛围一致性，尤其在高 γ 值时差异更为明显。

图 1（文首效果展示）以完整对比呈现了本文方法的整体迁移效果。

4.6.2 提示词的影响

在定性实验中，我们观察到提示词质量会明显影响生成结果。使用简单提示词时，模型对内容理解有限，细节匮乏；使用详细提示词时，模型能更准确理解用户意图，风格一致性和视觉质量通常更好。这表明在风格迁移任务中，文本提示是引导扩散模型理解内容语义的重要因素。

4.7. 实验中的关键发现

4.7.1 Canny边缘图分辨率与推理时间的权衡

Canny边缘图的精细程度直接影响生成效果和推理效率。表 9测试了不同Canny分辨率下的推理时间。

表 9. Canny分辨率与推理时间对比。高分辨率Canny边缘图可提供更丰富的结构信息从而提升生成效果，但推理时间明显增加，且精细边缘图可能引入额外的人工痕迹。

输入分辨率	Canny分辨率	推理时间	说明
1080×599	1080×599	约12秒	标准分辨率
5774×1536	5774×1536	约68秒	高分辨率

关键发现：(1) 更精细的Canny边缘图能提升生成效果，模型获得更丰富的结构信息；(2) 但精细边缘图可能导致

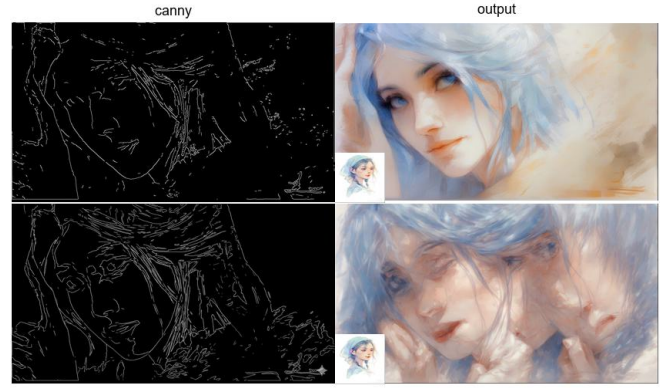


图 14. 不同Canny分辨率下的结构条件与生成结果对比。左列为Canny边缘图，右列为对应风格化输出，输出图左下角嵌入参考风格图。

模型过度响应局部边缘，产生描边感、碎片化纹理或不自然的局部细节；(3) 推理时间随Canny分辨率明显增加；(4) 需在过于精细（人工痕迹多、开销大）和过于粗糙（丢失细节）之间找平衡。如图 14所示，不同分辨率下Canny边缘检测的精细程度差异显著。基于这些观察，前端支持用户动态调节Canny边缘检测参数。

4.7.2 设备资源限制

扩散模型（Stable Diffusion基础模型约7-8GB，加上VAE、ControlNet以及特征缓存等模块，总显存需求约15-20GB）在消费级GPU上运行仍有一定挑战。即使使用V100-32GB，批量生成时也面临显存压力和GPU资源竞争等问题。相比在更多block或更多processor中进行注入，本文在`up_blocks.0.attentions.1`内采用选择性processor注入（默认仅5个self-attention processor），从而控制额外计算和缓存开销，使整个流程在V100上能够流畅运行。

5. 交互式前端

为提升实际可用性和用户体验，我们基于Gradio框架构建了名为**StyleKV Control Studio**的交互式前端。

5.1. 前端功能模块

(1) 图像输入模块：内容图像上传、风格图像上传、自动生成并预览Canny边缘图。

(2) 提示词控制模块：正向提示词（描述期望生成结果）、反向提示词（不希望出现的元素），支持长文本输入。

(3) **ControlNet/Canny**参数调节：Canny Low/High Threshold（边缘检测灵敏度）、Conditioning Scale（ControlNet控制强度）、Bilateral Filter参数（边缘预处理去噪）。

(4) **AdaIN**风格融合模块：AdaIN Alpha（融合强度）、AdaIN Beta（融合偏置），支持实时预览。

(5) 生成参数控制：Steps（DDIM采样步数，默认50）、CFG Scale（无分类器引导，默认7.5）、Seed（随机种子）、KV融合强度 γ 。

(6) 输出管理：输出目录设置、生成状态实时显示、结果画廊展示。

5.2. 前端特性与价值

前端支持参数热重载（调整参数后无需重启即可查看效果）、Canny边缘图实时预览、配置保存为JSON文件、批

量生成。设计目标是降低使用门槛，使非专业用户也能通过直观的滑块、开关和预览图理解各参数对生成结果的影响。图 15展示了该前端的整体界面布局。

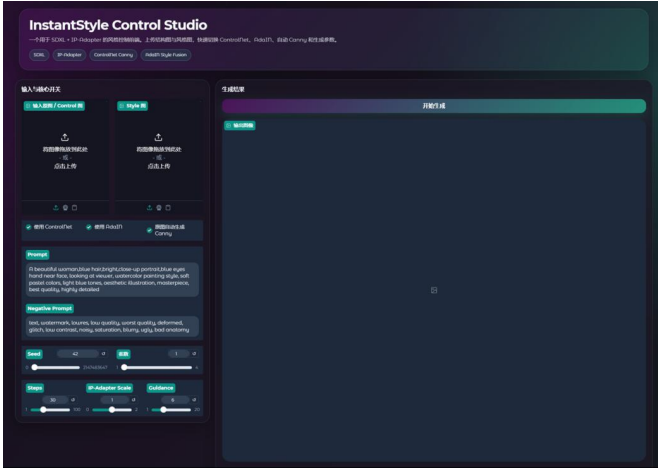


图 15. StyleKV Control Studio交互式前端界面。用户可通过左侧面板上传内容与风格图像、调节ControlNet/Canny参数、设置AdaIN融合系数及生成参数；右侧区域实时展示Canny边缘图预览与风格化输出结果。

6. 未来工作

6.1. 与现有方法的定量比较

当前实验聚焦于自身消融研究以验证各组件有效性，尚未与InST[12]、StyleDiffusion[13]、InstantStyle[17]等方法进行同等条件下的定量比较。计划在统一的数据集和评估协议下进行系统性的方法间对比。

6.2. 自适应Canny边缘图生成

当前需手动调节Canny阈值，且最优值因图像内容而异。计划引入HED、RINDNet等更优边缘提取模型，实现自适应边缘图生成，减少手动调参需求。

6.3. VLM提示词自动生成

当前提示词需人工编写或借助外部模型生成。计划引入开源视觉语言模型（如Qwen-VLM、LLaVA）自动为风格图像生成高质量提示词，降低使用门槛。

6.4. 跨架构适配与轻量化

本文方法在SDXL上验证，其有效注意力处理器的选择与融合策略与具体架构相关。计划将KV融合机制适配到SD1.5、SD3等其他扩散架构，重新识别各架构下的有效self-attention processor，并探索蒸馏与量化等轻量化方案，降低对高显存设备的依赖。

6.5. 引入Tile ControlNet

高分辨率Canny边缘图可能引入人工痕迹。计划使用Tile ControlNet通过分块处理缓解高分辨率下的人工痕迹问题，同时保持细节丰富度。

6.6. 处理器级贡献度分析

当前仅测试1/2个目标block的naive AdaIN配置，以及up_blocks.0.attentions.1内1/5/10个self-attention processor的StyleKV配置。计划逐个测试U-Net中不同self-attention processor的风格迁移贡献度，建立处理器-风格

映射关系，为方法优化提供理论依据。

6.7. 多条件ControlNet融合

当前仅集成Canny ControlNet。计划探索多个ControlNet（边缘+深度+姿态等）的协同工作，进一步提升内容保持能力。

7. 结论与局限性

7.1. 结论

本文提出了一种无需训练的AdaIN StyleKV风格迁移方法。主要贡献包括：

- 方法创新：提出风格图像KV与内容图像KV的加权融合策略，实现自注意力层特征的精确操控；将AdaIN扩展到文本特征与图像特征的跨模态条件融合中，通过对齐嵌入特征的通道统计量改善色彩协调性与条件一致性。
- 系统验证：在COCO+WikiArt数据集上构建7组方法变体，生成5000张图像进行组件消融、方法变体对比与参数敏感性分析。实验表明完整方法在风格保真度与内容保留之间实现了较优平衡。
- 实践价值：基于Gradio构建交互式前端，支持Canny调节、AdaIN参数控制、实时对比等功能。
- 经验总结：详细记录了实验中的关键发现，包括提示词质量的重要性、Canny分辨率与推理时间的权衡等。

7.2. 局限性

尽管本文方法取得了较好效果，仍存在以下局限性，如表 10所示。这些局限性同时也是第6节未来工作的重点攻关内容。

表 10. 本文方法的主要局限性及对应改进方向。

局限性	说明	对应改进方向
缺乏与外部SOTA方法的对比	仅进行自身变体消融，未与InST、StyleDiffusion、InstantStyle等方法在统一基准下直接比较	在统一数据集与评估协议下复现并定量对比主流方法
处理器级消融不足	仅测试up0.attn1内少量self-attention processor配置，未系统验证所有self-attention processor	处理器级贡献度分析
ControlNet单一提示词依赖外部评价维度有限	仅测试Canny，其他条件未集成；依赖外部模型生成，未集成开源VLM；主要依赖FID、Art-FID和LPIPS，尚未覆盖更多感知与语义指标	多条件ControlNet融合；VLM提示词自动生成；引入更多评价指标并补充人工主观验证
设备资源受限	需V100级别GPU，消费级设备难以运行	模型轻量化探索

7.3. 未来展望

风格迁移是一个充满活力的研究领域，随着扩散模型的不断发展，以下方向值得持续关注：(1) 免训练方法的进一

步优化, 在保持内容结构的同时实现更精确的风格控制; (2) 多模态风格的融合, 结合文本描述、参考图像、色彩调色板等多种条件; (3) 风格迁移的可解释性, 理解U-Net各层在风格迁移中的角色; (4) 轻量化部署, 将方法压缩并部署到移动端。

致谢

感谢所有对本文工作提供支持和帮助的老师 and 同学。感谢AutoDL平台提供的GPU计算资源支持。感谢开源社区贡献的Stable Diffusion、ControlNet等优秀工作, 为本文的研究提供了坚实的基础。

References

- [1] A. A. Efros and T. K. Leung, “Texture synthesis by non-parametric sampling,” in *Proc. ICCV*, 1999.
- [2] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image style transfer using convolutional neural networks,” in *Proc. CVPR*, 2016.
- [3] D. Ulyanov, V. Lebedev, A. Vedaldi, and V. Lempitsky, “Texture networks: Feed-forward synthesis of textures and stylized images,” in *Proc. ICML*, 2016.
- [4] X. Huang and S. Belongie, “Arbitrary style transfer in real-time with adaptive instance normalization,” in *Proc. ICCV*, 2017.
- [5] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, “Unpaired image-to-image translation using cycle-consistent adversarial networks,” in *Proc. ICCV*, 2017.
- [6] Y. Choi, M. Choi, M. Kim, J.-W. Ha, S. Kim, and J. Choo, “StarGAN: Unified generative adversarial networks for multi-domain image-to-image translation,” in *Proc. CVPR*, 2018.
- [7] T. Karras, S. Laine, and T. Aila, “A style-based generator architecture for generative adversarial networks,” in *Proc. CVPR*, 2019.
- [8] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep unsupervised learning using nonequilibrium thermodynamics,” in *Proc. ICML*, 2015.
- [9] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Proc. NeurIPS*, 2020.
- [10] Y. Song and S. Ermon, “Improved techniques for training score-based generative models,” in *Proc. NeurIPS*, 2020.
- [11] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proc. CVPR*, 2022.
- [12] Y. Zhang, N. Huang, F. Tang, H. Huang, C. Ma, W. Dong, and C. Xu, “InST: Inversion-based style transfer with diffusion models,” in *Proc. CVPR*, 2023.
- [13] Z. Wang, L. Zhao, W. Xing, and D. Lu, “StyleDiffusion: Controllable disentangled style transfer via diffusion models,” in *Proc. ICCV*, 2023.
- [14] J. Jeong, M. Kwon, and Y. Uh, “DiffStyle: Training-free style transfer with diffusion models,” *arXiv preprint arXiv:2312.01714*, 2023.
- [15] L. Zhang, A. Rao, and M. Agrawala, “Adding conditional control to text-to-image diffusion models,” in *Proc. ICCV*, 2023.
- [16] A. Hertz, R. Mokady, J. Tenenbaum, K. Aberman, Y. Pritch, and D. Cohen-Or, “Style Injection: Training-free style transfer using self-attention,” *arXiv preprint arXiv:2312.08008*, 2023.
- [17] H. Wang, Q. Wang, X. Bai, Z. Qin, and A. Chen, “InstantStyle: Free lunch towards style-preserving in text-to-image generation,” *arXiv preprint arXiv:2404.02733*, 2024.
- [18] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *Proc. ECCV*, 2014.
- [19] WikiArt, “WikiArt Visual Art Encyclopedia.” [Online]. Available: <https://www.wikiart.org/>.
- [20] M. Wright and B. Ommer, “ArtFID: Quantitative evaluation of neural style transfer,” in *Proc. GCPR*, 2022, arXiv:2207.12280.
- [21] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, “GANs trained by a two time-scale update rule converge to a local Nash equilibrium,” in *Proc. NeurIPS*, 2017.
- [22] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” in *Proc. CVPR*, 2018.
- [23] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *Proc. ICLR*, 2015.
- [24] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual losses for real-time style transfer and super-resolution,” in *Proc. ECCV*, 2016.
- [25] S. Liu, T. Lin, D. He, F. Li, M. Wang, X. Li, Z. Sun, Q. Li, and E. Ding, “AdaAttN: Revisit attention mechanism in arbitrary neural style transfer,” in *Proc. ICCV*, 2021.
- [26] Y. Deng, F. Tang, W. Dong, C. Ma, X. Pan, L. Wang, and C. Xu, “StyTr²: Image style transfer with transformers,” in *Proc. CVPR*, 2022.
- [27] J. An, S. Huang, Y. Song, D. Dou, W. Liu, and J. Luo, “ArtFlow: Unbiased image style transfer via reversible neural flows,” in *Proc. CVPR*, 2021.

- [28] H. Chen, L. Zhao, Z. Wang, H. Zhang, Z. Zuo, A. Li, W. Xing, and D. Lu, “DualAST: Dual style-learning networks for artistic style transfer,” in *Proc. CVPR*, 2021.
- [29] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, and T. Aila, “Analyzing and improving the image quality of StyleGAN,” in *Proc. CVPR*, 2020.
- [30] T. Karras, M. Aittala, S. Laine, E. Härkönen, J. Hellsten, J. Lehtinen, and T. Aila, “Alias-free generative adversarial networks,” in *Proc. NeurIPS*, 2021.
- [31] M. Lei, X. Song, B. Zhu, H. Wang, and C. Zhang, “StyleStudio: Text-driven style transfer with selective control of style elements,” in *Proc. CVPR*, 2025.

附录

A. 7组变体完整数据

表 11列出了7组变体的完整实验数据，与正文表5一致。

表 11. 7组变体完整定量数据（与正文表5一致）。

变体名称	FID↓	Art-FID↓	LPIPS
V1 (adain_1blk)	73.34	103.17	0.581
V2 (adain_2blk)	88.54	98.42	0.599
V3 (no_adain_1blk)	76.57	98.63	0.597
V4 (no_adain_2blk)	91.56	95.02	0.612
V5 (adain_no_ctrl)	70.75	104.12	0.746
V6 (proc05_no_adain)	78.14	85.43	0.629
V7 (proc05_adain)	78.48	90.76	0.599

B. Gamma消融实验数据

表 12列出了 γ 消融实验的完整数据。

表 12. γ 消融实验完整数据（`up_blocks.0.attentions.1`内5个self-attention processor + AdaIN + ControlNet，与正文表6一致）。

γ	FID↓	Art-FID↓	LPIPS
0.0	76.57	98.63	0.597
0.2	76.91	94.26	0.606
0.4	77.46	89.72	0.617
0.6	78.48	90.76	0.599
0.8	79.67	83.58	0.655
1.0	82.36	82.91	0.697